

ESP32 AirTag Scanner — Complete Hardware Guide

Firmware: ESP32 AirTag Scanner · **Upstream:** [MatthewKuKanich/ESP32-AirTag-Scanner](#) · CYD touchscreen fork: [hevnsnt/CYD_ESP32-AirTag-Scanner](#) **Chips:** ESP32, ESP32-S3 (CYD variant is ESP32 + ILI9341 touchscreen) · **Cyber Controller profile:** [airtag-scanner](#) (esptool backend, merged single-bin @ 0x0, no suicide) **This guide:** purchase → build → flash → integrate into Cyber Controller → use → troubleshoot.

1. Overview

ESP32 AirTag Scanner is a **counter-surveillance BLE detector**. It puts the ESP32 radio into a continuous Bluetooth Low Energy scan and surfaces the **Apple FindMy advertisements** that AirTags (and FindMy-compatible beacons) broadcast — so you can tell whether an unknown tracker may be travelling with you. The base firmware is **serial/UART-only**: it prints each detected tag's MAC address, RSSI (signal strength, a rough proximity cue), and the raw advertisement payload to a 115200-baud terminal — no Android phone or nRF Connect app needed (§9). A separate community fork adds an on-device **2.8" touchscreen** UI for the CYD board (§3).

Cyber Controller flashes the firmware via esptool and reads its serial stream in the **Devices** tab, so the tag list appears in the persistent terminal alongside your other connected hardware. Note the upstream tool is a **detector** — it reports what it hears; any *interaction* with a tracker (e.g. forcing a lost AirTag to play a sound) is **not** documented in the upstream README (verify against the source before relying on it — §9).

2. Legal & Safety

This firmware is a **passive receiver** — it only listens to BLE advertisements that nearby devices already broadcast in the clear. Passive scanning for trackers as a **personal-safety / anti-stalking** measure is broadly lawful and is the firmware's intended use: detecting a tracker someone may have planted on you or your vehicle. Keep it lawful: - **Detect, don't weaponize.** Use it to find trackers *on you/your property*. Do not use captured MAC/payload data to surveil, follow, or identify other people — that can cross into stalking/harassment law. - **It is not a court instrument.** RSSI is a coarse proximity hint, not proof of who placed a tracker. If you believe you are being stalked, preserve the device and contact local law enforcement. - **Radio rules still apply.** Even passive RF monitoring is regulated in some jurisdictions — check local law. - Related projects that *transmit* spoofed FindMy beacons (e.g. AirTag clones) are a different, **offensive** category and are out of scope for this detector; do not conflate them.

3. Hardware & Purchasing

Per the Cyber Controller profile the firmware targets three board classes. Pick by whether you want a standalone screen or a serial-only sensor driven from Cyber Controller / a Flipper.

Tier	Board	Chip	Screen	Why	Where to buy (search terms)
Cheapest / serial-only	ESP32-WROOM-32 DevKit	esp32 (4MB)	none	Lowest cost; read tags in Cyber Controller's terminal	AliExpress/Amazon: "ESP32 DevKit V1" / "ESP32-WROOM-32 devkit"
More BLE headroom / native USB	ESP32-S3 DevKitC	esp32s3 (8MB)	none	Newer radio, native-USB COM port, more flash	AliExpress/Amazon: "ESP32-S3 DevKitC-1"
Standalone with display	CYD "Cheap Yellow Display" ESP32-2432S028R	esp32 (4MB)	2.8" 320×240 ILI9341 resistive touch	On-device tag readout, no PC needed — requires the hevnsnt fork	AliExpress/Amazon: "ESP32-2432S028R CYD 2.8"
Flipper companion	any ESP32-WROOM / ESP32-S3 module	esp32 / esp32s3	none	Flash via Flipper ESP Flasher, read on Flipper UART terminal	Flipper store / "ESP32 WiFi dev board"

Notes & accessories: - The **base MatthewKuKanich firmware has no screen** — output is serial only. A display is only present on the **CYD fork**, which drives the ILI9341 panel via the TFT_eSPI library (§4, §9). - Buy a **data-capable USB cable** (many cheap cables are charge-only → no COM port). - Identify your USB-serial chip before buying drivers: most CYD boards use **CH340**; many DevKits use **CP210x**; ESP32-S3 boards usually present a **native USB** port. (Verify on the specific listing.) - Verify flash size matches the profile: **4MB** for ESP32/CYD, **8MB** for the ESP32-S3 board. Avoid clones with undersized flash. - An external antenna only helps on boards that expose a u.FL connector; most listed boards use a PCB antenna.

4. Building / Assembly

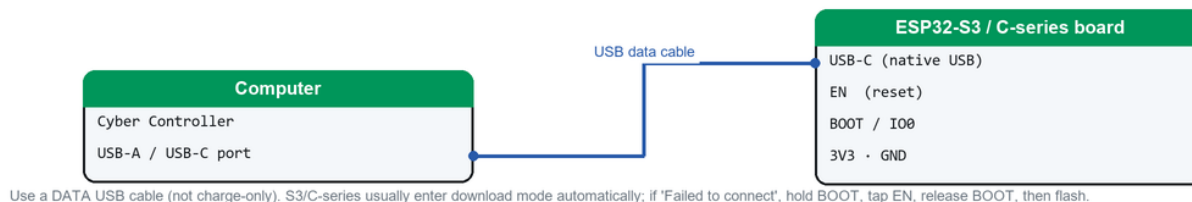
There is **no physical assembly** for any supported board — each is a complete dev board needing only a USB data cable. "Building" here means producing the firmware image, because the upstream repo currently ships **no formal release binaries** (the Releases page is empty — §9). Two paths:

- **Pre-built bins (Flipper path):** the repo includes the three Arduino-export binaries `airtag_scanner.ino.bootloader.bin`, `airtag_scanner.ino.partitions.bin`, and `airtag_scanner.ino.bin`, intended for the Flipper Zero **ESP Flasher** app (Apps → GPIO → ESP Flasher → Manual Flash). These map to bootloader / partition table / FirmwareA respectively.
- **Build from source (Arduino IDE):** open `AirTag_Scanner.ino` in Arduino IDE, install ESP32 board support, select your board (e.g. *ESP32 Dev Module* / *ESP32S3 Dev Module*), set upload speed **115200**, and Upload.
- **CYD fork extra steps:** install the **TFT_eSPI** library and copy the fork's provided `User_Setup.h` into the library so the ILI9341 panel + touch are configured; install **CH340** drivers first; then upload with any ESP32 board profile (*ESP32 Dev Module* works). (verify: exact `User_Setup.h` values ship with the fork — §9.)

If you want Cyber Controller to flash a single merged image (its preferred model, §5), you can export/merge a combined `.bin` from your build, or wait for an upstream release. The profile's flash core tolerates a missing release and falls back to "source only" rather than erroring.

5. Flashing & First Run (via Cyber Controller)

Flashing connection — ESP32 (native USB)



How to connect the board to flash it via Cyber Controller (native-USB ESP32).

1. Connect the board by USB; open Cyber Controller → **Flash** tab.
2. **Port:** pick the board's COM/tty (click *Refresh* if missing → USB-serial driver issue, see §8).
3. **Firmware Profile:** ESP32 *AirTag* Scanner (profile id `airtag-scanner`, `esptool` backend).
4. **Board / chip:** choose **ESP32 Generic** (`esp32`, 4MB), **ESP32-S3 Generic** (`esp32s3`, 8MB), or **ESP32 CYD 2.8" Touchscreen** (`esp32 + ILI9341`). The profile maps any release asset whose name contains `s3` to `esp32s3`, everything else to `esp32`.
5. Click **Flash**. Cyber Controller writes a **merged single image at offset 0x0** at **115200** baud. - **If the upstream release is empty/404** (it currently is — §9): the resolver returns no asset and the flash core uses its source-only fallback rather than failing. In that case flash a locally built/merged `.bin` (§4), or use the Flipper ESP Flasher three-file method.
6. **First run:** after flashing, press the board's **RESET** button and open a serial terminal at **115200** (Cyber Controller's terminal, or the Flipper UART Terminal app). The device starts scanning and prints detected AirTags — MAC, RSSI (dBm), and hex payload. On the **CYD fork** the 2.8" screen shows the same tag readout instead of (or in addition to) serial.

6. Integrate into Cyber Controller

- **Profile:** airtag-scanner (esptool; image_model: merged-single-bin; app_offset 0x0; supports_suicide: false). Chips: esp32, esp32s3. Resolver = GitHub release latest, asset match *.bin, chip from filename fragment (s3 → esp32s3, else esp32); on_error: source_only_empty so a missing release is non-fatal.
- **Control:** open the **Devices** tab → select the port → set **Firmware = ESP32 AirTag Scanner** → **Connect** at **115200** baud. The profile has **no protocol-aware parser** (protocol: null), so output appears as raw serial text in the persistent terminal — read the MAC/RSSI/payload lines directly rather than expecting them to auto-populate the Targets pool.
- **Commands:** the firmware is mostly autonomous (scans on boot). It accepts a serial **rescan** command to reset detection state and re-list tags from scratch — type it in the Devices terminal. (verify: exact command token in the source — §9.)
- **No dangerous verbs:** this is a passive detector, so there are no attack/transmit commands and **Dead Man's Switch / suicide is not applicable** (supports_suicide: false).
- **Backup first:** if you're reflashing a board that already runs other firmware, use **Backup** in the Flash tab to dump existing flash before overwriting.

7. Usage (end-to-end)

1. **Power on / connect:** plug in the board; in Cyber Controller open **Devices**, select the port, Firmware = *ESP32 AirTag Scanner*, **Connect** at 115200 (or open the Flipper UART Terminal). On a CYD, just power it.
2. **Watch the stream:** the device prints a line per detected FindMy advertisement — sequential count, **MAC address**, **RSSI (dBm)**, and the raw hex **payload**. More-negative RSSI = farther; closer to 0 = nearer.
3. **Sweep for a planted tracker:** walk away from / around your space. A tracker *on you* keeps a steady or rising RSSI as you move, while ambient tags (neighbors, passers-by) drop off. Move the board around a bag, car, or jacket to triangulate by signal strength.
4. **Re-scan:** send the **rescan** command (or RESET the board) to clear the seen-list and re-enumerate — useful after you've moved to a new location.
5. **Confirm a hit responsibly:** if a tag persistently tracks you, treat it as a personal-safety matter (§2) — preserve the device, do not attempt to surveil whoever owns it.
6. **Caveats:** the base firmware detects **Apple FindMy / AirTag** advertisements; it does **not** reliably distinguish *paired vs separated/lost* AirTags, and broader tracker types (Samsung SmartTag, Tile) are **not** confirmed in the base source — the CYD fork's README claims them, and upstream issue #3 tracks the request. (verify per build — §9.)

8. Troubleshooting

- **No COM port:** install the right USB-serial driver — **CH340** (most CYD boards), **CP210x** (many DevKits), or native USB (ESP32-S3); swap in a **data** cable; on Linux add your user to `dialout` and replug.
- **Flash fails / "Failed to connect":** enter bootloader manually — hold **BOOT**, tap **RESET**, release **BOOT**, then flash; lower the flash baud; close any serial monitor holding the port.
- **No tags ever appear:** confirm the terminal is at **115200**; press **RESET** after flashing; make sure a real AirTag/FindMy device is actually nearby and broadcasting (a paired tag near its owner advertises less).
- **Blank CYD screen:** the display only works on the **hevnsnt CYD fork** with the correct **TFT_eSPI User_Setup.h** — the base firmware has no screen; reflash the fork build with the matching display config.
- **Cyber Controller shows "no release asset" / source-only:** expected — the upstream repo has **no published releases** (§9). Provide a locally built/merged `.bin` (§4) or use the Flipper three-file ESP Flasher method.
- **Garbled serial text:** wrong baud (use 115200) or wrong USB-serial driver.
- **Boot-loops / suspected bad flash: Erase Flash** in the Flash tab, then reflash; verify the board's flash size matches the profile (4MB esp32 / 8MB esp32s3) and isn't an undersized clone.

9. Sources

- Upstream firmware: <https://github.com/MatthewKuKanich/ESP32-AirTag-Scanner> (README, `AirTag_Scanner.ino`, Issues, empty Releases page) — confirms ESP32-WROOM/ESP32-S3 support, Flipper ESP Flasher three-file flash, 115200 serial output of MAC/RSSI/payload, Apple FindMy (0x4C) detection, and no on-device display.
- CYD touchscreen fork: https://github.com/hevnsnt/CYD_ESP32-AirTag-Scanner — 2.8" 320×240 display, TFT_eSPI + `User_Setup.h`, CH340 drivers; fork README mentions AirTags/Samsung SmartTags/Tile (verify per build).
- Cyber Controller profile: `src/config/profiles/airtag_scanner.json` (id `airtag-scanner`; esptool; chips `esp32/esp32s3`; merged-single-bin @ 0x0; default baud 115200; `on_error: source_only_empty`; `supports_suicide: false`; CYD variant noted as the hevnsnt fork).
- Context / counter-stalking background: AirGuard, AirFlag, and security research on FindMy detection (general reference for the personal-safety use case).
- **Verify at use time:** the exact `rescan` command token, any tracker types beyond Apple AirTag, and whether the build offers any tracker *interaction* (upstream documents detection only) — confirm in the current source. Vendor links/board availability change; re-check before purchase.